

Reliable Data Engineering and Reproducibility for Large-Scale Pretraining

Yan Liu, Jiayi Ji, Xiaohan Wang, Jinglun Mao, and Shengpeng Li

Abstract—Large-scale pretraining has shifted the center of model development from isolated model design to full-lifecycle data engineering. Web-scale corpora provide broad coverage, but they also contain noise, duplication, bias, benchmark contamination, private information, and poorly documented transformations. These defects affect model capability, evaluation credibility, legal compliance, and the cost of reproducing experiments. This paper synthesizes recent work on reliable pretraining data engineering. We first summarize common quality issues in large-scale pretraining data, then review cleaning, deduplication, filtering, selection, and quality assessment methods. We further discuss governance mechanisms, including dataset versioning, lineage tracking, metadata management, and compliance control. Finally, we conduct a small controlled case study to illustrate representative reproducibility risks that also exist in large-scale pretraining pipelines. The survey suggests that reliable pretraining can benefit from a unified workflow in which data quality control, governance, and reproducibility are managed together rather than treated as separate afterthoughts.

Index Terms—Large-scale pretraining, data quality, data processing, data governance, reproducibility, large language models.



1 INTRODUCTION

FOUNDATION models have become a central paradigm in modern artificial intelligence. Large language models such as GPT-3 demonstrate that increasing model scale can substantially improve in-context learning and few-shot generalization [1], while neural scaling laws relate model performance to parameter count, data volume, and compute budget [2]. Chinchilla further shows that, under a fixed compute budget, model size and training tokens must be scaled jointly [3]. These findings make pretraining data a first-order design variable rather than a passive resource. As foundation models are pretrained on broad and heterogeneous corpora before being adapted to many downstream tasks, defects introduced during pretraining may be inherited, amplified, and made difficult to diagnose later [4].

The data used for pretraining are now assembled from Common Crawl, Wikipedia, books, code repositories, academic papers, forums, and social media. Public corpora such as The Pile, C4, Dolma, and FineWeb illustrate both the value and difficulty of this process: they combine many sources, long pipelines, repeated filtering stages, and substantial variation in language, domain, and quality [5], [6], [7], [8]. Open model efforts such as BLOOM, Pythia, and OLMo also show that dataset transparency and released artifacts are essential for understanding model behavior beyond final benchmark scores [9], [10], [11].

Data quality problems appear throughout the pretraining lifecycle. Noisy webpages, duplicated documents, imbalanced language distributions, contaminated benchmarks, and personally identifiable information can reduce training efficiency, increase memorization, distort evaluation, and create privacy or compliance risks [12], [13], [14]. These

problems are not independent. For example, deduplication changes the effective distribution of domains; filtering may remove harmful text while also suppressing minority dialects; and benchmark leakage may spread through undocumented derived datasets. Reliable pretraining therefore calls for more than a sequence of local preprocessing operations.

This paper organizes reliable data engineering for large-scale pretraining into four connected layers and a concluding synthesis. Section 2 analyzes data quality issues. Section 3 reviews data processing methods. Section 4 discusses governance mechanisms that make data assets traceable and auditable. Section 5 examines reproducibility and engineering practice. Section 6 concludes with a unified view. The central claim is that pretraining reliability depends on a managed chain from raw data to final model: quality, governance, and reproducibility are better designed together.

The main contributions of this paper are as follows. First, we summarize five major quality risks in large-scale pretraining data, including noise, duplication, bias, benchmark contamination, and privacy leakage. Second, we review representative processing methods and discuss their trade-offs in terms of quality, diversity, coverage, safety, and efficiency. Third, we connect data governance mechanisms with reproducibility requirements and argue for a lifecycle view of pretraining data engineering. Fourth, we provide a small controlled case study on TinyStories to illustrate how randomness, implementation choices, and software environments affect reproducibility in practice.

2 DATA QUALITY ISSUES

Large-scale pretraining data are usually assembled from open, weakly curated sources. Many defects are therefore built into the collection process rather than appearing as accidental outliers. The most common issues are noise, duplication, bias, benchmark contamination, and privacy

- Y. Liu, J. Ji, X. Wang, J. Mao, and S. Li are with the School of Computer Science and Technology, Tongji University, Shanghai, China.
E-mail: y_an@tongji.edu.cn; 2246903540@qq.com;
christinewxh1221@gmail.com; 1744458515@qq.com;
nuleboyefev928@gmail.com.
- All authors contributed equally to this work.

leakage. These issues affect different parts of the model life-cycle: noise and duplication reduce training efficiency, bias changes the distribution of learned behavior, contamination weakens evaluation validity, and leakage creates safety and compliance risks. Because the same document may contain several of these problems at once, quality analysis must consider both sample-level defects and dataset-level distribution shifts.

2.1 Noise and Low-Value Text

Noise refers to text that contributes little useful training signal or actively interferes with learning. Typical examples include invalid hyperlinks, boilerplate navigation, advertisements, spam, repeated templates, garbled characters, OCR errors, broken markup, machine-translated fragments, and low-readability documents. Reviews of massive web-mined corpora emphasize that such defects are common rather than exceptional [15]. In crawled corpora, noisy content is often reported as a substantial fraction of raw data, which means that even a small error rate in the filter can leave billions of low-value tokens.

The effect of noise is both computational and statistical. Computationally, noisy samples consume memory bandwidth and optimization steps without providing stable semantic structure. Statistically, corrupted or incoherent text can make the model allocate capacity to artifacts that do not generalize. Severe noise may also increase loss variance because the training objective is asked to predict irregular symbols, broken formatting, or unrelated fragments. This problem is especially visible when model developers use the same raw Web source across many experiments: the defects become a persistent background factor that is difficult to attribute after training.

Noise is difficult to remove completely because it is mixed with useful content at document, paragraph, and line granularity. A webpage may contain a high-quality article surrounded by navigation menus and advertising text; a forum thread may contain valuable domain knowledge written in informal spelling; a software repository may include both useful examples and generated files. Consequently, filtering rules must distinguish low-value artifacts from nonstandard but meaningful language. The challenge is not only the presence of noise, but also the risk of deleting legitimate data when cleaning becomes too aggressive.

2.2 Duplication and Memorization

Duplication appears when crawlers repeatedly collect the same page, mirror sites redistribute identical content, or derived datasets include overlapping sources. It can be divided into exact duplication and near duplication. Exact duplicates are identical after normalization and can be detected through hashes. Near duplicates share most of their content but differ in templates, timestamps, comments, markup, or small edits. Near duplication is more important at Web scale because syndicated articles, reposted code, copied tutorials, and benchmark mirrors frequently differ only in surface details.

Deduplication matters because repeated samples change the effective sampling distribution. If a document appears many times, the optimizer treats it as more important than

its source prevalence would justify. Lee et al. show that deduplicating training data improves language models by reducing memorization and improving generalization [12]. In large corpora, even modest duplicate ratios represent billions of repeated tokens and significant wasted computation. Repetition can also increase the probability that a model memorizes rare strings, including benchmark examples or private information.

Duplication is not merely a storage issue. Repeated documents may overrepresent particular domains, authors, viewpoints, or benchmark-like patterns. Longpre et al. show that training data age, domain coverage, quality, and toxicity interact with model performance in nontrivial ways [13]. From a practitioner's perspective, high-quality datasets should balance correctness, diversity, coverage, information density, and readability rather than optimize a single scalar quality score [16]. Deduplication therefore needs to be evaluated together with coverage: removing duplicates should reduce memorization without unintentionally eliminating useful minority-domain data.

2.3 Bias and Coverage Imbalance

Bias in pretraining data arises from imbalance across languages, regions, cultures, demographics, domains, and source platforms. Since web-scale data are not sampled from a carefully designed population, English and highly visible Web communities are often overrepresented, while low-resource languages, local knowledge, and marginalized groups may be underrepresented. These imbalances can be transferred into model outputs, creating unequal performance across user groups and systematic differences in generated content. Surveys on trustworthy LLMs connect such data bias with broader problems of hallucination, unfairness, and safety [17].

Coverage imbalance also affects capability. A model trained on a narrow distribution may perform well on popular domains but poorly on specialized, multilingual, or culturally specific tasks. Language imbalance can reduce fluency and factuality in low-resource languages; demographic imbalance can change how occupations, identities, and social roles are associated; domain imbalance can make the model strong on general Web prose but weak on scientific, legal, medical, or regional material. These effects are difficult to correct after pretraining because they are embedded in representation learning.

The challenge is that diversity and quality can conflict under naive filtering. If a classifier is trained to prefer polished English reference corpora, it may label informal, multilingual, or local texts as low quality even when they are valuable for downstream robustness. Bias mitigation must therefore be tied to measurement. Teams need to monitor source composition, language distribution, demographic representation, toxicity, cultural perspective, and domain coverage before and after each processing step. A reliable pipeline should report not only the final dataset size, but also how filtering changed the groups represented in the corpus.

2.4 Benchmark Contamination and Evaluation Leakage

Benchmark contamination occurs when evaluation examples, labels, or closely paraphrased versions enter the pre-

training corpus. As data collection expands across public repositories, educational websites, discussion forums, dataset mirrors, and model-evaluation discussions, this risk becomes difficult to avoid. Open contamination reports and surveys show that leakage can appear in many forms, including input-only contamination and input-label contamination [18], [19]. Kernel-divergence based measurements further indicate that contamination can distort benchmark interpretation [20].

Contamination undermines the meaning of evaluation. A model may answer an item because it has memorized the example rather than because it has learned the underlying reasoning skill. Dong et al. frame this as a central distinction between generalization and memorization in trustworthy evaluation [21]. Input-label contamination is especially damaging because the model may learn the exact mapping from question to answer. Input-only contamination is less direct but still problematic because familiarity with the prompt distribution can inflate apparent performance.

The practical consequence is that evaluation pipelines must include benchmark decontamination, leakage audits, and clear reporting of matching methods, thresholds, and inspected sources. Exact string matching is insufficient for paraphrased or reformatted items, while overly broad semantic matching may create false positives. Reliable evaluation therefore combines exact matching, normalized n-gram matching, embedding-based similarity, and manual inspection for high-risk benchmarks. Without these records, a high score may reflect data management failure rather than model capability.

2.5 Privacy and Sensitive Information

Large web corpora often include names, email addresses, phone numbers, addresses, credentials, medical details, and other personally identifiable information. PII-Scope provides systematic evidence that private information can be present in pretraining data and may be extractable from pre-trained models [14]. This creates security risks for deployed systems and legal risks for model providers, especially when data were scraped without explicit consent or when sensitive records are retained after transformation.

Privacy leakage differs from ordinary noise because its cost is asymmetric. A small number of leaked records may be enough to harm individuals or violate regulation. Moreover, privacy information can be hidden inside otherwise useful documents, such as email archives, forum posts, support tickets, code comments, or scraped personal pages. A filter that removes entire sources may sacrifice useful content, while a filter that only redacts obvious patterns may miss contextual private information.

Therefore, privacy filtering should be treated as a mandatory safety control rather than an optional quality enhancement. Effective systems combine pattern-based detection, learned classifiers, manual audit samples, source-level exclusion rules, and post-training extraction tests. They also need provenance records so that, when a problematic source is identified, derived datasets and downstream models can be traced. Privacy risk is thus directly connected to governance: without lineage and version records, deletion requests and incident response become difficult to execute reliably.

TABLE 1
Major Processing Methods for Pretraining Data

Method	Main purpose	Main risk
Cleaning	Remove noisy, unsafe, malformed, or low-readability text	Aggressive rules may reduce coverage or erase minority language patterns
Deduplication	Remove exact, near, or semantic duplicates	Threshold choices may distort domain balance
Filtering and selection	Prioritize samples with high training utility	Proxy models may import reference-corpus bias
Quality assessment	Measure corpus and sample quality before and after processing	Single scores may hide diversity and subgroup effects

3 DATA PROCESSING METHODS

Data processing transforms raw crawls into trainable corpora. It is often described as preprocessing, but at pre-training scale it is closer to a production data system: each operator changes the corpus distribution, affects training cost, and creates artifacts that must be recorded. Table 1 summarizes the main method families and their risks.

3.1 Data Cleaning

Data cleaning is usually the first stage of the pretraining data pipeline. Rule-based cleaning removes documents using language identification, length thresholds, repetition ratios, URL density, special-character proportions, stop-word statistics, and boilerplate removal. These methods are efficient enough for terabyte- or petabyte-scale corpora and remain necessary even when more advanced classifiers are available. They are also interpretable: when a document is removed because it is too short, contains too many symbols, or fails language identification, the decision can be audited.

FineWeb demonstrates industrial-scale practice in which heuristic filtering, deduplication, and quality classifiers are combined to produce high-quality Web data at very large scale [8]. Dolma similarly uses a multi-stage pipeline to construct an open three-trillion-token English corpus [7]. Data-Juicer generalizes these practices into a modular processing system with reusable operators, analysis modules, and feedback loops between data processing and model training [22]. These systems show that cleaning is no longer a single script; it is a configurable pipeline whose decisions must be reproducible.

However, cleaning is not neutral. Saada et al. argue that many quality classifiers learn to match the distribution of reference datasets rather than intrinsic quality [23]. Mansour and Heckel show that web-filtered datasets can retain measurable fingerprints of their filtering pipelines and that such bias may propagate through training [24]. Domain-specific corpora require localized rules. Mangosteen, for example, adapts processing to Thai rather than blindly applying Western-centric heuristics [25]. The general lesson is that cleaning pipelines should report both what they remove and how the remaining corpus changes.

3.2 Data Deduplication

Deduplication methods can be divided into exact, approximate, and semantic approaches. Exact deduplication computes hashes over documents or normalized spans and removes identical copies. It is simple, scalable, and useful as a first pass, but it cannot detect repeated content with minor edits. Approximate deduplication typically uses MinHash and locality-sensitive hashing to estimate set similarity and identify near duplicates. This family is widely used because it scales to billions of documents while preserving an interpretable similarity threshold.

LSHBloom combines MinHash signatures with Bloom filters to reduce disk use and improve throughput for extreme-scale document deduplication [26]. FED accelerates deduplication with GPU-based signature generation and hashing, showing that deduplication itself has become a major systems problem for trillion-token corpora [27]. Snapshot-aware strategies, such as those used in FineWeb, help distinguish within-snapshot duplication from cross-snapshot repetition [8]. This distinction matters because repeated content within one crawl may indicate boilerplate, while repeated content across time may also represent persistent and socially important information.

The choice of deduplication threshold is consequential. A strict threshold may leave paraphrased or template-based duplicates, while a loose threshold may merge legitimately related but distinct documents. Semantic deduplication further uses embeddings, sentence encoders, or LLM-assisted judgments to detect repeated meaning beyond surface overlap. Comparative work on semantic similarity methods suggests that embedding-based techniques can identify latent duplicates missed by traditional string matching [28]. For pretraining, the practical target is not only fewer duplicates, but also a better effective sampling distribution.

3.3 Filtering and Selection

Filtering removes undesirable data, whereas selection actively prioritizes data with high expected training utility. Data selection methods include heuristic rules, proxy language models, influence or importance resampling, task-oriented density ratios, and multi-objective optimizers. DSIR estimates the density ratio between candidate data and target-domain data, turning selection into a distribution matching problem [29]. Surveys of language-model data selection emphasize that the field is moving from static preprocessing toward adaptive selection guided by downstream objectives and efficiency constraints [30].

Proxy model-based filtering uses lightweight language models, perplexity models, or scoring classifiers to rank candidate samples. Compared with rules, these models capture richer semantic signals such as coherence, informativeness, and educational value. The trade-off is proxy bias: a scoring model inherits the preferences of its architecture, training data, and reference labels. Task-oriented selection reduces this problem by defining utility relative to a target distribution, but it may overfit a narrow downstream objective.

Multi-dimensional integration methods address these limitations by combining heterogeneous quality signals. FIRE integrates multiple quality ratings into pretraining data selection [31], while QuaDMix jointly balances quality

and diversity [32]. These studies support a multi-objective view: the best dataset is not necessarily the one with the highest average quality score, but the one that preserves the right mixture of useful, diverse, safe, and efficient samples. Multilingual settings make this issue sharper. Ali et al. show that quality judging across languages requires language-specific adaptation rather than direct transfer of English-centric filters [33].

3.4 Data Quality Assessment

Data quality assessment supplies the measurements used by cleaning, deduplication, and selection. At the dataset level, teams track language composition, source distribution, document length, vocabulary richness, near-duplicate rate, toxicity, benchmark overlap, domain coverage, and temporal coverage. At the sample level, they use perplexity, readability, classifier scores, source reputation, language-model judgments, and manual audit labels. These measurements should be collected before and after each major processing stage so that the effect of the stage can be observed.

Assessment is most useful when it is comparative rather than absolute. A single quality score cannot express whether a dataset has become less diverse, less multilingual, or less representative after filtering. Dataset-level dashboards should therefore report distributional changes and subgroup metrics alongside aggregate quality. Sample-level quality scores should be calibrated with manual audits and downstream training experiments. This is especially important for low-resource languages, technical domains, and informal user-generated text, where generic perplexity or readability scores may penalize valuable data.

3.5 Pipeline-Level Trade-Offs

Processing components should not be optimized independently. Cleaning removes noise and harmful content, deduplication mitigates redundancy and memorization risks, filtering and selection prioritize training utility, and quality assessment provides quantitative guidance. Yet aggressive cleaning may sacrifice knowledge coverage; classifier-based filtering can import reference-corpus bias; deduplication can change domain balance; and excessive emphasis on quality may weaken diversity and generalization. Effective pretraining data processing is therefore a multi-objective optimization problem that jointly balances quality, diversity, coverage, safety, and computational efficiency.

4 DATA GOVERNANCE MECHANISMS

Processing methods improve data, but governance makes the improvement auditable and sustainable. In large-scale pretraining, pointwise cleaning is insufficient because data sources, processing code, compliance requirements, and model releases evolve over time. Governance provides the institutional and technical layer that connects dataset versions, provenance, access control, and legal obligations. Table 2 summarizes the main governance functions.

TABLE 2
Governance Functions in Pretraining Data Pipelines

Function	Records	Supported decision
Version control	Dataset hashes, source snapshots, processing parameters	Which data produced a model or metric
Lineage tracking	Source, transformation, derivative links	Where a defect or contamination path originated
Compliance control	Licenses, consent status, privacy checks, access logs	Whether data may be used, shared, or deleted
Metadata management	Datasheets, model cards, audit notes	How others interpret and reproduce results

4.1 Data Version Control

Datasets should be managed as first-class assets with versions, metadata, access policies, and change histories. Traditional Git is poorly suited to storing terabytes or petabytes of raw data, but the principles of content-addressed versioning remain important. Conceptual work on data versioning emphasizes that a useful version must record not only changed files, but also acquisition time, processing context, and the meaning of the change [34]. For LLMs, this implies that model cards and experiment logs should record dataset version identifiers, source snapshots, filtering configurations, deduplication thresholds, and hash values.

Practical version-control systems often separate metadata from bulk storage. Git may track code, configuration files, and small pointer files, while the actual data are stored in object stores, dataset hubs, or distributed file systems. Tools such as DVC follow this pattern by recording content hashes and metadata in a code repository while keeping large artifacts in remote storage. Hugging Face Datasets and dataset hubs provide commit-based release identifiers, while experiment trackers such as MLflow can associate runs with data versions and configuration files. The common principle is that a training run should be linked to an immutable dataset identity, not to a vague file path or manually edited note.

Metadata is the bridge between version control and reproducibility. Datasheets for Datasets provide a structured way to document motivation, composition, collection process, preprocessing, distribution, and maintenance [35]. Leipzig et al. argue that metadata is central to reproducible computational research because it enables others to understand the data and software context behind results [36]. Recent work on LLM-oriented data management similarly identifies dataset maintenance, storage, documentation, and lifecycle control as bottlenecks for reliable model development [37].

4.2 Data Lineage and Provenance

Data lineage answers where a sample came from, what transformations it underwent, and which downstream artifacts depend on it. Without lineage, duplicated sources, contamination, and privacy problems can propagate through derived datasets without detection. A benchmark item included in one raw crawl may be duplicated, filtered, sampled into several mixtures, and then used by several model

releases. If this path is not recorded, later decontamination becomes guesswork.

Provenance is thus a practical requirement for accountability. Atlas applies supply-chain provenance ideas to the machine-learning lifecycle and records verifiable lineage metadata for data, code, and training artifacts [38]. This kind of end-to-end record matters because model harms often require tracing back to source data and processing decisions. Provenance also improves debugging: when an evaluation score changes unexpectedly, developers can compare not only model code but also upstream data transformations.

Automated provenance tools can reduce the burden of manual documentation. Gregori et al. propose an LLM-guided platform for collecting and managing multi-granular data provenance [39], while yProv4ML offers lightweight provenance tracking for machine-learning systems [40]. These tools point toward a governance pattern in which scripts, intermediate datasets, metrics, and configurations are captured during execution rather than reconstructed after failure. Metadata management for AI-augmented workflows and systems such as TableVault further show that dynamic, human-AI data creation requires explicit management of dataset state and transformation history [41], [42].

4.3 Compliance and Accountability

Compliance management connects technical governance to legal and ethical obligations. The EU AI Act requires transparency and technical documentation for high-risk AI systems, including information about training, validation, and testing data [43]. In China, generative AI service regulation and the broader cybersecurity, data security, and personal information protection framework require providers to consider data source legality, personal information protection, and content safety [44]. These obligations make dataset documentation, provenance, and deletion mechanisms operational necessities rather than academic preferences.

Compliance operates at several levels. Source-level review asks whether the data may be collected and used. Sample-level review asks whether personal information, copyrighted content, or sensitive material should be removed or transformed. Pipeline-level review asks whether transformations preserve required records, whether cross-border storage is allowed, and whether access control matches the data classification. Release-level review asks whether model cards, data statements, and risk assessments are sufficient for external users. These layers must be connected because a legal decision about a source may need to propagate to every derived dataset and model.

Governance also supports internal accountability. If a model is found to memorize private information, produce harmful bias, or show suspicious benchmark performance, teams need to know which dataset version, source snapshot, and filtering configuration were used. A mature governance system therefore links compliance checks with version control, lineage graphs, access control, audit logs, and model release documentation. The same records that support legal review also support scientific reproducibility and engineering debugging.

5 REPRODUCIBILITY AND ENGINEERING PRACTICE

5.1 Reproducibility as a Multi-Layer Property

The previous sections focus on how data are collected, processed, and governed; reproducibility asks whether the resulting claims can be reconstructed and interpreted. Unlike small deterministic algorithms, modern pretraining pipelines involve massive datasets, stochastic optimization, GPU kernels, distributed communication, mixed precision, and complex software environments. Even when the model architecture and hyperparameters are fixed, training results may still vary because of random initialization, data order, numerical reduction order, or dependency versions. Studies on random seeds and nondeterminism show that small implementation details can change optimization trajectories and reported metrics [45], [46], [47], [48]. Broader reproducibility guidance in machine learning also emphasizes that code, data, environment, and execution metadata must be reported together [49], [50].

Reproducibility should therefore be studied not only as a final-metric problem, but also as an engineering process involving traceable data, code, environment, and runtime configurations. It is useful to distinguish three levels. Bit-wise reproducibility means identical weights, losses, and intermediate states; this is the strictest form and is often expensive on GPUs and distributed systems. Statistical reproducibility means repeated runs produce comparable means, variances, and trends; this is often the practical target for large-scale pretraining. Engineering reproducibility means the experiment can be reconstructed from recorded code, data, environment, configuration, and commands. Open projects such as Pythia and OLMo approximate this higher standard by releasing checkpoints, data-loading tools, training details, and evaluation code [10], [11].

5.2 Experimental Setup

We conduct a small controlled case study using a character-level Transformer on a public TinyStories text slice [51]. The goal is not to claim state-of-the-art performance or to generalize directly to frontier-scale models, but to illustrate representative reproducibility risks that also exist in large-scale pretraining pipelines. TinyStories is suitable for this purpose because it is public, lightweight, and sufficiently structured to produce meaningful language-model training curves under limited compute.

We use the public TinyStories validation split only as a lightweight text source. It is not used here as an official benchmark validation set. The selected text slice consists of the first 2,500,000 bytes and is further divided into an internal 90%/10% train-validation split for the controlled reproducibility study. The recorded SHA256 hash of the local text slice is `c56c22686ca95c441e8754db4daa8fe45f6112c52e2fffd45e5968bc589357485`. The model contains token and positional embeddings, three Transformer encoder layers, hidden size 96, four attention heads, a causal attention mask, and a final linear prediction head over the character vocabulary. The dataset is converted into character IDs. Each batch samples contiguous blocks of length 64 and predicts the next character. Table 3 summarizes the training configuration.

TABLE 3
Training Configuration for the Reproducibility Case Study

Parameter	Value
Training steps	80
Block size	64
Global batch size	64
Learning rate	3×10^{-4}
Optimizer	AdamW
Evaluation interval	Every 10 steps
Metrics	Validation loss, perplexity, tokens/s

This experiment is intentionally small and controlled. It does not aim to reproduce frontier-scale pretraining or to claim that the measured numerical gaps directly transfer to large models. Instead, it serves as an illustrative case study for isolating three engineering factors: random seeds, implementation choices under the same global batch size, and software-environment dependencies. These factors are easier to observe and document in a lightweight setting, while the same categories of risks are commonly amplified in larger distributed pretraining systems.

To control randomness, the implementation fixes Python, NumPy, PyTorch, and CUDA seeds. Deterministic mode controls cuDNN deterministic behavior, cuDNN benchmark mode, Flash attention, memory-efficient attention, and `torch.use_deterministic_algorithms()`. The setting `CUBLAS_WORKSPACE_CONFIG=:4096:8` is also used to reduce cuBLAS-related nondeterminism.

5.3 E1: Randomness Ablation

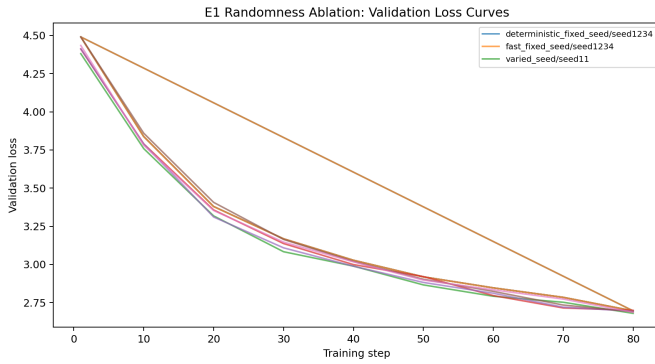
Randomness enters pretraining through weight initialization, data sampling, dropout, GPU kernels, and sometimes distributed scheduling. E1 isolates this factor by keeping the dataset, model, and hyperparameters fixed while changing only the seed and deterministic settings. Table 4 reports both the experimental design and the aggregate results. The varied-seed group uses five seeds: 11, 22, 33, 44, and 55.

Figure 1 shows that fixed-seed curves nearly overlap, indicating highly reproducible optimization trajectories when random sources are controlled. The varied-seed runs diverge visibly, showing that initialization and sampling order can change the training trajectory even under identical data and hyperparameters. The fixed-seed runs are exactly reproducible in this setup. Changing the seed introduces a final-loss standard deviation of 0.0066 and a perplexity standard deviation of about 0.097. The fast fixed-seed setting achieves higher throughput than the deterministic setting, suggesting that stricter determinism may come with a performance cost.

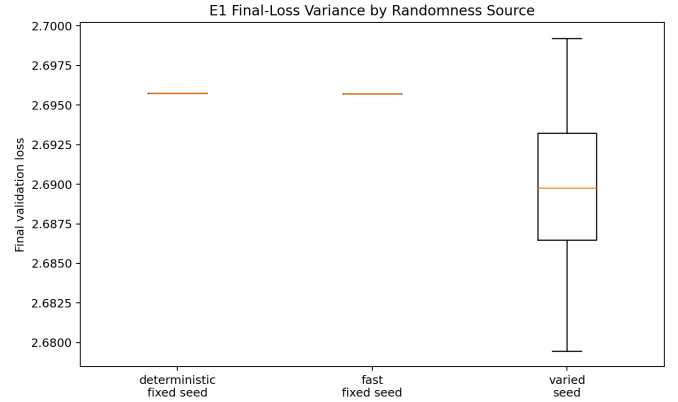
The implication is that single-run results are insufficient for reproducibility analysis. A rigorous pretraining report should include random seeds, repeated runs, means, standard deviations, and training curves. Reporting only the best or latest single result can hide seed sensitivity and overstate stability.

TABLE 4
E1 Randomness Ablation: Design and Results

Configuration	Description	Runs	Mean final loss	Std. final loss	Mean perplexity	Tokens/s
deterministic_fixed_seed	Fixed seed with deterministic PyTorch/CUDA settings	2	2.6957	0.0000	14.82	115260
fast_fixed_seed	Fixed seed with faster non-strict CUDA/cuDNN execution paths	2	2.6957	0.0000	14.82	135009
varied_seed	Different random seeds to estimate seed-induced variance	5	2.6896	0.0066	14.73	136357



(a)



(b)

Fig. 1. Randomness ablation results: (a) validation loss curves under randomness settings and (b) distribution of final validation loss. Fixed-seed curves nearly overlap, while varied seeds produce visible trajectory and final-loss dispersion.

5.4 E2: Equal Global Batch and Gradient-Accumulation Simulation

Distributed pretraining often uses data parallelism, gradient accumulation, mixed precision, and collective communication. Even when the effective global batch size is unchanged, different implementations may not follow exactly the same numerical path. Because the available server has one RTX 3090 GPU, E2 uses micro-batch gradient accumulation to approximate one numerical issue in distributed training. Table 5 shows the configurations and results.

This design approximates a core numerical issue in distributed training: gradients are computed in partitions and then aggregated, while floating-point addition is not strictly associative. Thus, reduction order and numerical precision can affect the exact trajectory. Figure 2 shows validation loss curves, final-loss deltas relative to `full_batch_fp32`, and throughput. The curves are very close, suggesting statistical equivalence, but they are not required to be bitwise identical. The final losses are nearly identical, but full-batch training is about 3.4–3.7 times faster than gradient accumulation because accumulation requires multiple forward and backward passes per effective global batch.

E2 demonstrates that equal global batch size does not guarantee identical training processes. In real multi-GPU pretraining, NCCL all-reduce, reduction order, mixed precision, and parallelism strategies may introduce small numerical differences. Statistical reproducibility is often a more realistic target than bitwise reproducibility, and reproducibility analysis should include both numerical behavior and engineering cost.

5.5 E3: Software Environment Dependency

Large-scale pretraining depends on a deep software stack. Python, PyTorch, CUDA, cuDNN, MKL, GPU drivers, and package versions can all affect execution behavior. E3 tests whether changing the software stack affects results or throughput when data, model, random seed, and hyperparameters are fixed. Table 6 compares the two environments and reports the final results.

During the setup of Env-B, PyTorch initially failed to import because of an MKL/OpenMP compatibility issue: `ImportError: libtorch_cpu.so: undefined symbol: iJIT_NotifyEvent`. The issue was fixed by downgrading MKL/OpenMP to a compatible version. This illustrates that environment dependencies affect not only final metrics, but also whether an experiment can be executed reliably.

Figure 3 shows that raw validation loss curves from the two environments almost overlap, indicating statistically consistent behavior. The magnified curve-difference plot reveals only small transient deviations during training, while the final-loss bars and zoomed axis show that both environments reach the same final-loss levels: 2.716218 in deterministic mode and 2.716126 in fast mode. The magnified final-loss deltas are on the order of 10^{-7} , about $+3.0 \times 10^{-7}$ for deterministic mode and -2.4×10^{-7} for fast mode relative to Env-A under the same configuration. Figure 4 shows a clearer engineering effect: Env-A is faster under both deterministic and fast settings, while Env-B is slower by about 3.9% in deterministic mode and about 6.6% in fast mode. Thus, environment dependence may not appear as a large metric degradation; instead, it can show up as changes in training speed, installation stability, and reproduction cost.

TABLE 5
E2 Equal Global Batch and Gradient-Accumulation Simulation: Design and Results

Configuration	Micro-batch	Accum. steps	Accum. order	Precision	Final loss	Perplexity	Tokens/s
full_batch_fp32	64	1	forward	fp32	2.6684	14.42	118187
accum4_forward_fp32	16	4	forward	fp32	2.6683	14.42	31522
accum4_reverse_fp32	16	4	reverse	fp32	2.6682	14.41	32439
accum4_forward_bf16	16	4	forward	bf16	2.6683	14.41	35152

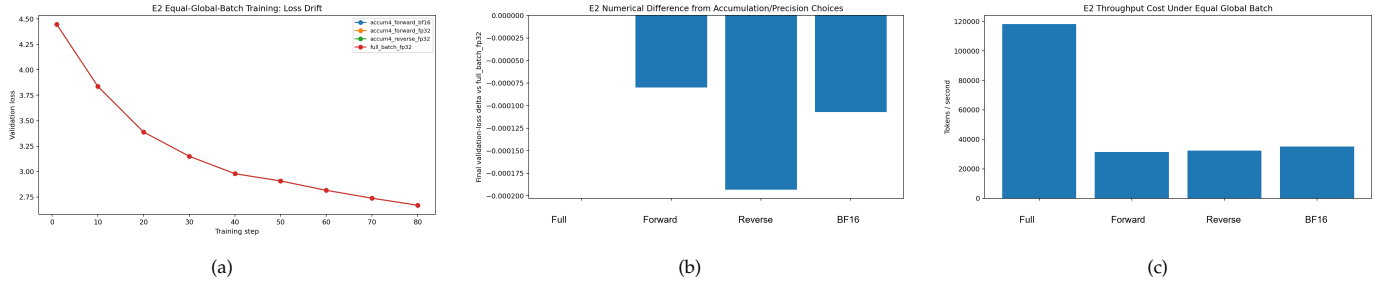


Fig. 2. Equal-global-batch simulation: (a) validation loss curves, (b) final-loss deltas, and (c) training throughput. The final losses are statistically close, but throughput differs substantially between full-batch training and gradient accumulation.

TABLE 6
E3 Software Environment Dependency: Environments and Results

Env.	Python	PyTorch	CUDA	cuDNN	GPU
Env-A	3.12.3	2.8.0+cu128	12.8	91002	RTX 3090
Env-B	3.10.20	2.4.1	12.1	90100	RTX 3090

Env.	Configuration	Runs	Final loss	Perplexity	Tokens/s
Env-A	env_deterministic_fp32	2	2.716218	15.12	118946
Env-A	env_fast_fp32	2	2.716126	15.12	136021
Env-B	env_deterministic_fp32	2	2.716218	15.12	114309
Env-B	env_fast_fp32	2	2.716126	15.12	127002

Values are representative final results; repeated runs check stability under each environment and configuration.

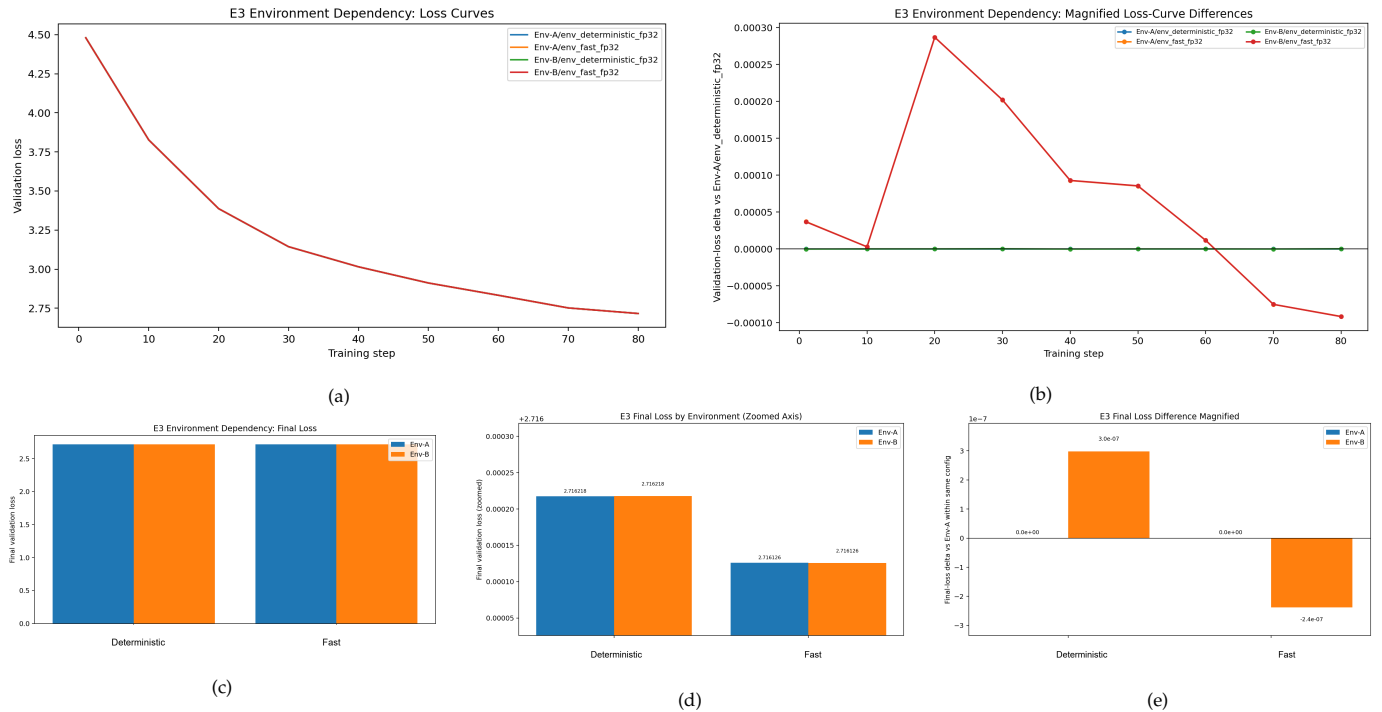


Fig. 3. Software-environment effects on validation loss: (a) raw curves, (b) magnified loss-curve deltas, (c) final validation loss, (d) zoomed final validation loss, and (e) magnified final-loss deltas. Raw curves and final-loss bars nearly overlap; magnified plots show differences only at very small scales.

5.6 Cross-Experiment Discussion and Engineering Recommendations

Taken together, the three experiments analyze reproducibility at different levels. Table 7 summarizes the main

observations and the artifacts that should be tracked in a rig-

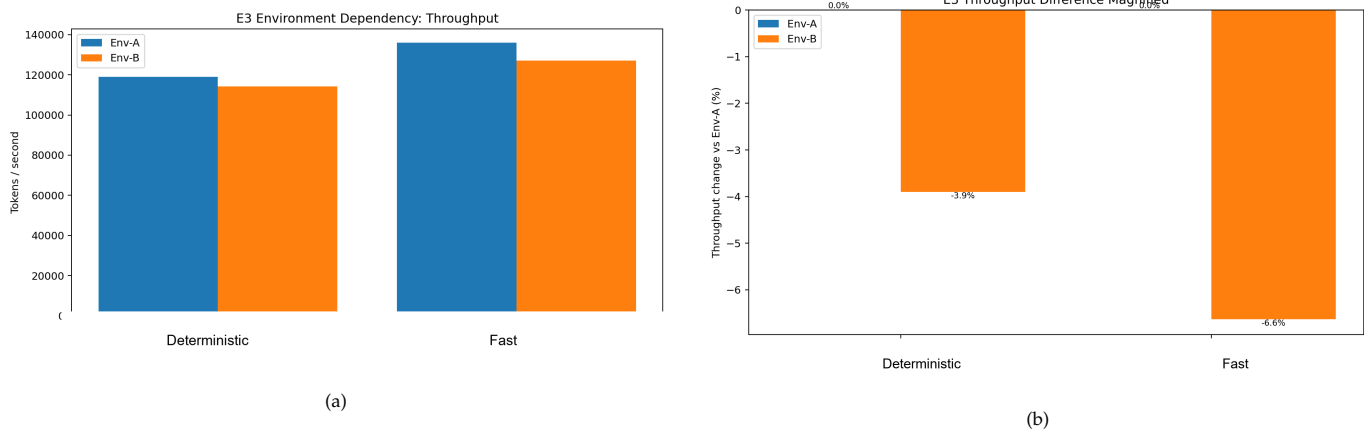


Fig. 4. Software-environment effects on throughput: (a) raw throughput and (b) throughput change relative to Env-A. Env-B is slower by about 3.9% in deterministic mode and about 6.6% in fast mode.

orous pretraining report. Random seeds affect optimization trajectories; equivalent global batch sizes can still differ in numerical path and throughput; and software-stack changes may preserve final metrics while changing execution speed and setup reliability. In large-scale pretraining, these issues are amplified by data order, dropout, mixed precision, all-reduce order, CUDA kernel selection, and dependency versions.

This artifact-tracking protocol connects reproducibility with engineering practice. Data hashes and versions make quality interventions traceable. Lineage and provenance explain how a training set was produced. Compliance records document why sources were included or excluded. Repeated-run statistics and environment metadata explain what variation remains after the data are fixed. Therefore, reproducibility should be treated as a combination of result stability, process traceability, and engineering cost rather than as a single final metric.

5.7 Implications for Reliable Pretraining Pipelines

The preceding analysis suggests that reliable pretraining pipelines should be organized as closed-loop systems rather than linear preprocessing chains. In a linear chain, raw data are collected, cleaned, deduplicated, filtered, mixed, trained on, and then archived. This structure is easy to implement, but it makes later diagnosis difficult because model behavior is observed only after many irreversible data decisions have already been made. A closed-loop pipeline records intermediate data states, measures distributional changes after each operation, and uses model-side signals to revise data-side assumptions. For example, a spike in memorized benchmark-like outputs should trigger contamination audits of the corresponding source snapshots; poor multilingual performance should trigger language- and region-level coverage analysis; and privacy extraction failures should be traced back through lineage records to the raw sources and filters that allowed sensitive strings to remain.

In practice, this closed-loop view can be implemented through release gates. Before a dataset version is approved for pretraining, the pipeline should pass checks for source legality, license status, language balance, near-duplicate rate, PII exposure, benchmark overlap, and basic readability

statistics. Before a model checkpoint is released, the corresponding training run should be linked to immutable data hashes, code commits, environment metadata, and repeated-run summaries. These gates do not eliminate risk, but they make risk explicit and reviewable. They also prevent quality control from depending on personal memory or informal spreadsheets, which become unreliable once projects involve multiple contributors, datasets, and compute environments.

Another implication is that data quality should be evaluated at multiple time scales. Short-term quality control focuses on whether a specific corpus is clean enough for the next training run. Medium-term maintenance focuses on whether new source snapshots change the distribution of language, domain, toxicity, or duplication. Long-term governance focuses on whether data can still be explained, licensed, deleted, and reproduced after model releases. These time scales require different evidence. Short-term decisions need fast statistics and sampled audits; medium-term decisions need dashboards and regression tests; long-term decisions need durable metadata, provenance graphs, and documentation. Treating all three as part of the same pipeline makes the system more resilient to both technical failures and regulatory change.

An operational pipeline can therefore separate evidence into several connected records. The source record describes where each collection comes from, what permission or license applies, when it was acquired, and which version of the source snapshot is being used. The transformation record describes the concrete processing operators, parameter settings, thresholds, and intermediate hashes that turn raw data into trainable text. The evaluation record describes quality scores, duplication rates, decontamination checks, privacy scans, and distributional summaries before and after major interventions. The training record links the approved dataset version to model configuration, random seeds, software environment, hardware, throughput, and repeated-run statistics. These records should not be treated as paperwork after the fact. They are the evidence that makes later claims about quality, safety, and reproducibility inspectable.

This record structure also changes how teams respond

TABLE 7
Cross-Experiment Findings and Recommended Artifact Records

Level or artifact	Main factor or required record	Observation or purpose
Randomness	Seeds, initialization, sampling order	Varied seeds introduce measurable final-loss variance
Training implementation	Micro-batch size, accumulation order, precision	Final metrics are close, but throughput differs substantially
Environment	Python, PyTorch, CUDA, cuDNN, MKL	Final metrics remain close, but throughput and setup stability change
Data	Dataset name, URL, slice range, SHA256 hash	Bind results to an identifiable training corpus
Code	Git commit, script path, configuration files	Reconstruct implementation state
Randomness	Python/NumPy/PyTorch/CUDA seeds, sampler settings	Explain repeated-run variance
Environment	Python, PyTorch, CUDA, cuDNN, GPU, driver, Conda/pip dependencies	Reconstruct execution context and diagnose setup failures
Training configuration	Batch size, micro-batch size, accumulation, precision, optimizer, learning rate	Compare numerical path and engineering cost
Results	Loss curves, final loss, perplexity, throughput, means and standard deviations	Report stability rather than a single best run
Reproduction instructions	Full commands, output directories, dependency-fix notes	Make reruns auditable and repeatable

to failures. If evaluation reveals benchmark memorization, the team should not only remove overlapping documents; it should identify the affected source snapshots, update the contamination test, rerun the relevant filters, and record the changed dataset hash. If privacy extraction exposes sensitive content, the response should include source-level review, filter adjustment, deletion or quarantine of affected records, and documentation of the remediation path. If multilingual or domain-specific performance regresses, the response should compare the changed corpus mixture with earlier approved versions rather than only adjusting model hyperparameters. In each case, the model symptom becomes a data-engineering investigation, and the fix becomes part of the reproducible history of the system.

The same principle applies to threshold selection. Data filters often rely on heuristic cutoffs for language identification confidence, document length, perplexity, toxicity, deduplication similarity, or classifier-based quality scores. A cutoff that improves average validation loss may still remove minority dialects, technical formats, or low-resource languages. Conversely, a permissive cutoff may preserve diversity while leaving more noise and privacy risk. Therefore, thresholds should be justified by paired measurements: one set describing model utility and another describing coverage, risk, and distributional change. When these measurements disagree, the decision should be recorded as an engineering trade-off rather than hidden inside a preprocessing script.

For large datasets, full manual inspection is impossible, but sampling can still make the pipeline accountable. Stratified samples from each source, language, domain, and filtering bucket can expose failures that aggregate metrics hide. Error categories discovered by sampling can then be converted into automated checks, making the pipeline gradually more precise. This process resembles regression testing in software engineering: every discovered data failure should leave behind a small test, metric, or audit rule that makes recurrence easier to detect. Over time, the data pipeline accumulates institutional memory, and new dataset

releases become easier to compare with previous ones.

Reporting practice should reflect the same engineering logic. A paper or technical report should not only state that a model was trained on a cleaned corpus; it should describe the major source families, the order of transformations, the approximate amount of data removed at each stage, the criteria used to retain or discard documents, and the evidence used to judge that the resulting corpus remained diverse. For reproducibility, the report should also state which artifacts are public, which are restricted because of licensing or privacy constraints, and how a restricted artifact can still be audited through hashes, statistics, or controlled access. This distinction is important because many large-scale datasets cannot be redistributed in full, yet their construction process can still be documented with enough precision to support independent scrutiny.

Cost should also be reported as part of reproducibility. Two pipelines may produce similar final validation loss but require very different storage, preprocessing time, GPU throughput, or human review effort. The experiments above show this point at a small scale: software environments and accumulation strategies can preserve final loss while changing runtime behavior. In real pretraining systems, these costs influence which data controls are practical. For instance, exact deduplication is cheaper than semantic deduplication, sampled privacy review is cheaper than exhaustive inspection, and lightweight language identification is cheaper than model-based document scoring. A reliable report should make these trade-offs visible so that later work can understand whether a method is not only effective but also maintainable.

This perspective also helps align responsibilities across roles. Dataset engineers maintain ingestion, cleaning, and versioning. Model engineers monitor training stability, throughput, and checkpoint behavior. Governance reviewers examine source permissions, deletion requests, privacy risk, and documentation. Evaluation engineers audit contamination, distributional robustness, and benchmark validity. These roles are tightly connected: a failure in one

layer often appears first as a symptom in another. A shared artifact record gives the team a common object of discussion and reduces the chance that quality, compliance, and reproducibility are optimized separately.

Finally, reproducibility should be used as a design constraint for data engineering rather than as a final reporting exercise. If a dataset cannot be reconstructed, then its quality cannot be independently evaluated. If a training environment cannot be reconstructed, then reported improvements may reflect software drift rather than data improvements. If a benchmark cannot be decontaminated, then evaluation cannot be trusted. Our analysis suggests that reliable pretraining can benefit from treating data, code, configuration, environment, metrics, and governance decisions as connected versioned artifacts. The result is a pipeline in which model capability, safety, and accountability can be improved together instead of optimized in isolation.

5.8 Limitations

This study has several limitations. First, the empirical case study is conducted on a small character-level Transformer and a limited TinyStories text slice. Therefore, the numerical results should not be interpreted as direct estimates of reproducibility variance in frontier-scale LLM pretraining. Second, the distributed-training experiment is simulated through gradient accumulation on a single GPU, which cannot fully capture communication effects, parallelism strategies, or hardware heterogeneity in real multi-GPU clusters. Third, the environment comparison covers two software stacks on the same GPU, so it reflects representative dependency effects rather than an exhaustive environment benchmark. Despite these limitations, the experiment is useful as a controlled engineering demonstration: it isolates reproducibility factors that are often harder to diagnose in large systems and connects them with the broader data-governance discussion.

6 CONCLUSION

Large-scale pretraining depends on data systems whose complexity is comparable to the models they support. Noise, duplication, bias, contamination, and privacy leakage affect not only model performance but also evaluation credibility, deployment safety, and legal compliance. Cleaning, deduplication, filtering, selection, and assessment provide necessary technical controls, but they can also reshape the dataset distribution and introduce new biases if applied without measurement. Governance mechanisms such as versioning, metadata, lineage, and compliance management make these interventions more auditable. Reproducibility then helps close the loop by recording the code, data, environment, randomness, and runtime choices needed to interpret results.

The overall lesson is that reliable pretraining is a lifecycle problem. A trustworthy pipeline can benefit from treating data as a managed engineering asset, documenting meaningful transformations, evaluating both quality and diversity, auditing privacy and benchmark leakage, and reporting repeated-run statistics under explicit environment records. Future work should move toward adaptive data pipelines that optimize quality, coverage, safety, and efficiency jointly while preserving provenance and reproducibility by design.

REFERENCES

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [2] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [3] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022.
- [4] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill *et al.*, “On the opportunities and risks of foundation models,” *arXiv preprint arXiv:2108.07258*, 2021.
- [5] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima *et al.*, “The Pile: An 800GB dataset of diverse text for language modeling,” *arXiv preprint arXiv:2101.00027*, 2020.
- [6] J. Dodge, M. Sap, A. Marasović, W. Agnew, G. Ilharco, D. Groeneveld, M. Mitchell, and M. Gardner, “Documenting large webtext corpora: A case study on the colossal clean crawled corpus,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 1286–1305.
- [7] L. Soldaini, R. Kinney, A. Bhagia, D. Schwenk, D. Atkinson, R. Authur *et al.*, “Dolma: An open corpus of three trillion tokens for language model pretraining research,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 15 725–15 788.
- [8] G. Penedo, H. Kydliček, A. Lozhkov, M. Mitchell, C. Raffel, L. von Werra, and T. Wolf, “The FineWeb datasets: Decanting the web for the finest text data at scale,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 30 811–30 849.
- [9] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé *et al.*, “BLOOM: A 176B-parameter open-access multilingual language model,” *arXiv preprint arXiv:2211.05100*, 2022.
- [10] S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan *et al.*, “Pythia: A suite for analyzing large language models across training and scaling,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 2397–2430.
- [11] D. Groeneveld, I. Beltagy, P. Walsh, A. Bhagia, R. Kinney, O. Tafjord, A. H. Jha *et al.*, “OLMo: Accelerating the science of language models,” *arXiv preprint arXiv:2402.00838*, 2024.
- [12] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, “Deduplicating training data makes language models better,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 8424–8445.
- [13] S. Longpre, G. Yauney, E. Reif, K. Lee, A. Roberts, B. Zoph *et al.*, “A pretrainer’s guide to training data: Measuring the effects of data age, domain coverage, quality, and toxicity,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2024, pp. 3245–3276.
- [14] K. K. Nakka, A. Frikha, R. Mendes, X. Jiang, and X. Zhou, “PII-scope: A comprehensive study on training data privacy leakage in pretrained LLMs,” in *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. Mumbai, India: The Asian Federation of Natural Language Processing and The Association for Computational Linguistics, 2025, pp. 3731–3765.
- [15] M. Perlekiwicz and R. Poświata, “A review of the challenges with massive web-mined corpora used in large language models pre-training,” in *International Conference on Artificial Intelligence and Soft Computing*. Springer Nature Switzerland, 2024, pp. 153–163.
- [16] X. Yu, Z. Zhang, F. Niu, X. Xia, D. Lo, and J. Grundy, “What makes a high-quality training dataset for large language models: A practitioners’ perspective,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 656–668.
- [17] Z. Lin, S. Guan, W. Zhang *et al.*, “Towards trustworthy LLMs: A review on debiasing and dehallucinating in large language models,” *Artificial Intelligence Review*, vol. 57, no. 9, p. 1, 2024.

- [18] Y. Li, Y. Guo, F. Guerin *et al.*, "An open-source data contamination report for large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 528–541.
- [19] C. Deng, Y. Zhao, Y. Heng *et al.*, "Unveiling the spectrum of data contamination in language model: A survey from detection to remediation," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 16 078–16 092.
- [20] H. K. Choi, M. Khanov, H. Wei *et al.*, "How contaminated is your benchmark? quantifying dataset leakage in large language models with kernel divergence," *arXiv preprint arXiv:2502.00678*, 2025.
- [21] Y. Dong, X. Jiang, H. Liu *et al.*, "Generalization or memorization: Data contamination and trustworthy evaluation for large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 12 039–12 050.
- [22] D. Chen, Y. Huang, Z. Ma, H. Chen, X. Pan, C. Ge, D. Gao, Y. Xie, Z. Liu, J. Gao, Y. Li, B. Ding, and J. Zhou, "Data-Juicer: A one-stop data processing system for large language models," in *Companion of the 2024 International Conference on Management of Data*, 2024, pp. 120–134.
- [23] T. N. Saada, L. Bethune, M. Klein, D. Grangier, M. Cuturi, and P. Ablyn, "The data-quality illusion: Rethinking classifier-based quality filtering for LLM pretraining," *arXiv preprint arXiv:2510.00866*, 2025.
- [24] Y. Mansour and R. Heckel, "Measuring bias of web-filtered text datasets and bias propagation through training," in *Proceedings of the International Conference on Learning Representations*, 2025.
- [25] W. Phatthiyaphaibun, C. Udomcharoenchaikit, P. Singkorapoom, K. Pipatanakul, E. Chuangsuwanich, P. Limkonchotiwat, and S. Nutanong, "Mangosteen: An open thai corpus for language model pretraining," *arXiv preprint arXiv:2507.14664*, 2025.
- [26] A. Khan, R. Underwood, C. Siebenschuh, Y. Babuji, A. Ajith, K. Hippe *et al.*, "LSHBloom: Memory-efficient, extreme-scale document deduplication," *arXiv preprint arXiv:2411.04257*, 2024.
- [27] Y. Son, C. Kim, and J. Lee, "FED: Fast and efficient dataset deduplication framework with GPU acceleration," *arXiv preprint arXiv:2501.01046*, 2025.
- [28] D. You and S. P. Fraiberger, "Evaluating deduplication techniques for economic research paper titles with a focus on semantic similarity using NLP and LLMs," *arXiv preprint arXiv:2410.01141*, 2024.
- [29] S. M. Xie, S. Santurkar, T. Ma, and P. Liang, "Data selection for language models via importance resampling," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 34 201–34 227.
- [30] A. Albalak, Y. Elazar, S. M. Xie, S. Longpre, N. Lambert, X. Wang *et al.*, "A survey on data selection for language models," *arXiv preprint arXiv:2402.16827*, 2024.
- [31] L. Xu, X. Zhang, F. Duan, S. Wang, J. Wang, and X. Cai, "FIRE: Flexible integration of data quality ratings for effective pretraining," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 14 521–14 541.
- [32] F. Liu, W. Zhou, B. Liu, Z. Yu, Y. Zhang, H. Lin *et al.*, "QuaD-Mix: Quality-diversity balanced data selection for efficient LLM pretraining," *arXiv preprint arXiv:2504.16511*, 2025.
- [33] M. Ali, M. Brack, M. Lübbering, E. Wendt, A. G. Khan, R. Rutmann *et al.*, "Judging quality across languages: A multilingual approach to pretraining data filtering with language models," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 8870–8909.
- [34] J. Klump *et al.*, "Versioning data is about more than revisions: A conceptual framework and proposed principles," *Data Science Journal*, vol. 20, no. 1, p. 12, 2021.
- [35] T. Geburu, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. D. III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [36] J. Leipzig, D. Nüst, C. T. Hoyt, S. Soiland-Reyes, K. Ram, and J. Greenberg, "The role of metadata in reproducible computational research," *arXiv preprint arXiv:2006.08589*, 2020.
- [37] Z. Wang *et al.*, "Data management for large language models: A survey," *arXiv preprint arXiv:2312.01700*, 2023.
- [38] M. Spoczynski, M. S. Melara, and S. Szyller, "Atlas: A framework for ML lifecycle provenance and transparency," *arXiv preprint arXiv:2502.19567*, 2025.
- [39] L. Gregori, P. L. Lazzaro, M. Lazzaro, P. Missier, and R. Torlone, "An LLM-guided platform for multi-granular collection and management of data provenance," *Journal of Big Data*, vol. 12, no. 1, p. 187, 2025.
- [40] G. Padovani, V. Anantharaj, and S. Fiore, "yProv4ML: Effortless provenance tracking for machine learning systems," *SoftwareX*, vol. 31, p. 102298, 2025.
- [41] J. Zhao and S. Krishnan, "Metadata management for AI-augmented data workflows," *arXiv preprint arXiv:2508.06814*, 2025.
- [42] —, "TableVault: Managing dynamic data collections for LLM-augmented workflows," *arXiv preprint arXiv:2506.18257*, 2025.
- [43] European Union, "Regulation (EU) 2024/1689: Artificial intelligence act, article 13," Official legal text, 2024. [Online]. Available: <https://artificialintelligenceact.eu/article/13/>
- [44] Cyberspace Administration of China, "Interim measures for the management of generative artificial intelligence services," Government document, 2023. [Online]. Available: https://www.gov.cn/zhengce/zhengceku/202307/content_6891750.htm
- [45] D. Picard, "torch.manual_seed(3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision," *arXiv preprint arXiv:2109.08203*, 2021.
- [46] C. Summers and M. J. Dinneen, "Nondeterminism and instability in neural network optimization," in *Proceedings of the 38th International Conference on Machine Learning*, 2021, pp. 9913–9922.
- [47] M. Morin and M. Willetts, "Non-determinism in TensorFlow ResNets," *arXiv preprint arXiv:2001.11396*, 2020.
- [48] N. T. Bui, G. K. Savova, and L. Wang, "Assessing the macro and micro effects of random seeds on fine-tuning large language models," in *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. Mumbai, India: The Asian Federation of Natural Language Processing and The Association for Computational Linguistics, 2025, pp. 41–46.
- [49] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alche Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.
- [50] J. Shenouda and W. U. Bajwa, "A guide to computational reproducibility in signal processing and machine learning," *arXiv preprint arXiv:2108.12383*, 2021.
- [51] R. Eldan and Y. Li, "TinyStories: How small can language models be and still speak coherent english?" *arXiv preprint arXiv:2305.07759*, 2023.